

Notas de Eliminación de la Recursión

Diego Acuña

Mayo 2026

1. Introducción

El modelo imperativo puro no posee recursión como construcción primitiva.

Por el contrario, la computación funcional se basa esencialmente en definiciones recursivas.

Uno de los resultados fundamentales de teoría de lenguajes consiste en mostrar que la recursión puede eliminarse sistemáticamente utilizando únicamente:

- estado mutable,
- iteración,
- estructuras explícitas de control.

El problema central es entonces:

$$\text{CHI} \Rightarrow \text{Tail Recursion} \Rightarrow \text{IMP}$$

Estas notas estudian:

- recursión estructural,
- folds,
- acumuladores,
- tail recursion,
- simulación explícita del stack,
- traducción a IMP,
- preservación semántica.

2. Recursión estructural sobre listas

2.1. Estructura inductiva

Las listas poseen la forma inductiva:

$$\square \quad x : xs$$

Toda lista es:

- vacía,
- o un elemento seguido de otra lista.

2.2. Recursión primitiva

Definición 2.1 (Recursión primitiva). Una función definida por recursión estructural sobre listas posee la forma:

$$f[] = b$$

$$f(x : xs) = h\ x\ (f\ xs)$$

Ejemplo 2.2.

$$\begin{aligned}\text{sum}\ [] &= 0 \\ \text{sum}\ (x:xs) &= x + \text{sum}\ xs\end{aligned}$$

3. Expansión operacional

Evaluar:

$$\text{sum}\ [1,2,3]$$

produce:

$$1 + \text{sum}[2,3]$$

$$1 + (2 + \text{sum}[3])$$

$$1 + (2 + (3 + \text{sum}[]))$$

$$1 + (2 + (3 + 0))$$

Observación 3.1. Cada llamada recursiva deja operaciones pendientes que deben almacenarse implícitamente.

4. foldr

4.1. Definición

$$\begin{aligned}\text{foldr}\ h\ b\ [] &= b \\ \text{foldr}\ h\ b\ (x:xs) &= h\ x\ (\text{foldr}\ h\ b\ xs)\end{aligned}$$

Teorema 4.1 (Caracterización de la recursión estructural). *Toda función definida por:*

$$f[] = b$$

$$f(x : xs) = h\ x\ (f\ xs)$$

puede escribirse como:

$$f = \text{foldr}\ h\ b$$

Demostración. Por inducción estructural sobre listas.

Caso base:

$$\text{foldr } h \ b \ [] = b = f []$$

Paso inductivo:

$$\text{foldr } h \ b \ (x : xs) = h \ x \ (\text{foldr } h \ b \ xs)$$

Por hipótesis inductiva:

$$= h \ x \ (f \ xs) = f(x : xs)$$

□

5. Procesamiento direccional

5.1. foldr

Sea:

$$f = \text{foldr } h \ b$$

Entonces:

$$f[x_1, \dots, x_n] = h \ x_1 \ (h \ x_2 \ (\dots (h \ x_n \ b)))$$

o equivalentemente:

$$x_1 \otimes (x_2 \otimes (\dots (x_n \otimes b)))$$

Observación 5.1. foldr procesa listas de derecha a izquierda.

6. Tail recursion

6.1. Definición

Definición 6.1 (Tail recursion). Una función es tail recursive si toda llamada recursiva ocurre en posición terminal.

Forma típica:

$$f \ a \ [] = a$$

$$f \ a \ (x : xs) = f(h(a, x)) \ xs$$

6.2. Interpretación

La computación consiste únicamente en:

- actualizar parámetros,
- reiterar el mismo proceso.

Los parámetros funcionan como memoria explícita de la computación.

7. Método de acumuladores

7.1. Ejemplo: reversa

Versión estructural:

```
rev [] = []  
rev (x:xs) = rev xs ++ [x]
```

Versión con acumulador:

```
revAcc a [] = a  
revAcc a (x:xs) = revAcc (x:a) xs
```

y:

```
rev = revAcc []
```

Observación 7.1. El acumulador almacena explícitamente información que antes permanecía implícita en el stack recursivo.

8. foldl

8.1. Definición

```
foldl h a [] = a  
foldl h a (x:xs) =  
    foldl h (h a x) xs
```

8.2. Expansión

$$foldl\ h\ a\ [x_1, \dots, x_n]$$

produce:

$$(((a \otimes x_1) \otimes x_2) \dots) \otimes x_n$$

Teorema 8.1. *foldl* procesa listas de izquierda a derecha.

Demostración. Por inducción sobre la longitud de la lista. □

9. Ejemplo de equivalencia

Teorema 9.1. *La función:*

```
sumAcc a [] = a  
sumAcc a (x:xs) = sumAcc (a+x) xs
```

satisface:

$$sumAcc(a, xs) = a + sum(xs)$$

Demostración. Por inducción estructural sobre xs .

Caso base:

$$sumAcc(a, []) = a$$

y:

$$a + sum([]) = a + 0 = a$$

Paso inductivo:

$$sumAcc(a, x : xs) = sumAcc(a + x, xs)$$

Por hipótesis inductiva:

$$= (a + x) + sum(xs)$$

Por asociatividad:

$$= a + (x + sum(xs))$$

$$= a + sum(x : xs)$$

□

Corolario 9.2.

$$sum(xs) = sumAcc(0, xs)$$

10. Equivalencia entre recursión y tail recursion

Hasta aquí estudiamos transformaciones directas mediante acumuladores para funciones linealmente recursivas.

Sin embargo, muchas definiciones recursivas generales no son tail recursive y generan múltiples computaciones pendientes simultáneamente.

Por ejemplo:

$$fact(n) = n \cdot fact(n - 1)$$

o:

$$fib(n) = fib(n - 1) + fib(n - 2)$$

En ambos casos la llamada recursiva no ocurre en posición terminal.

La pregunta central es entonces:

¿Toda computación recursiva puede reformularse mediante tail recursion?

La respuesta es afirmativa si se permite representar explícitamente el estado restante de la computación.

Teorema 10.1 (Recursión y tail recursion). *Toda computación recursiva efectiva admite una formulación tail recursive equivalente mediante estructuras explícitas de control.*

Demostración. Una ejecución recursiva puede verse operacionalmente como una sucesión de estados:

(expresión actual, variables, stack)

El stack implícito almacena:

- llamadas pendientes,
- resultados parciales,
- puntos de retorno,
- operaciones aún no ejecutadas.

Por ejemplo:

$$n \cdot fact(n - 1)$$

deja pendiente la multiplicación por n .

Análogamente:

$$fib(n - 1) + fib(n - 2)$$

requiere recordar:

- el cálculo pendiente de la segunda rama,
- y posteriormente la suma de ambos resultados.

La idea fundamental consiste en representar explícitamente dichas computaciones pendientes mediante estructuras auxiliares.

En lugar de depender del stack implícito del runtime, se introducen:

- stacks explícitos,
- listas de pendientes,
- acumuladores,
- marcas de control.

Cada paso de evaluación:

- modifica explícitamente dichas estructuras,
- actualiza el estado,
- y continúa mediante una única llamada terminal.

La computación completa puede entonces reformularse de modo que:

- toda llamada recursiva ocurra en posición terminal,
- el control restante quede codificado explícitamente.

Por lo tanto, toda computación recursiva efectiva admite una formulación tail recursive equivalente. $\square$

11. Equivalencia entre tail recursion e iteración

Teorema 11.1 (Tail recursion e iteración). *Toda función tail recursive puede implementarse mediante un loop imperativo equivalente.*

Demostración. Considérese:

$$\begin{aligned}f(x) &= b(x) \\ f(x) &= f(h(x))\end{aligned}$$

La llamada terminal no requiere conservar contexto adicional.
Por lo tanto:

- los parámetros pueden almacenarse en variables mutables,
- la llamada recursiva puede reemplazarse por actualización de variables,
- el retorno puede reemplazarse por reiteración.

Obtenemos:

```
while !B(x) do
  x := h(x);

out := b(x);
```

que produce exactamente la misma sucesión de estados observables. □

12. Preservación semántica

Teorema 12.1 (Preservación semántica). *Sea:*

$$f \Rightarrow f_{TR} \Rightarrow p_{IMP}$$

la secuencia:

- *función recursiva,*
- *formulación tail recursive,*
- *programa IMP.*

Entonces:

$$\forall x \quad f(x) = f_{TR}(x) = p_{IMP}(x)$$

Demostración. La transformación:

$$f \Rightarrow f_{TR}$$

preserva el significado reemplazando el stack implícito por estructuras explícitas equivalentes.
La transformación:

$$f_{TR} \Rightarrow p_{IMP}$$

preserva exactamente las transiciones de estado reemplazando llamadas terminales por iteración.

Por lo tanto ambas transformaciones preservan semántica observacional. □

13. Estructuras no lineales

13.1. Árboles

```
data Tree a =  
  Leaf  
  | Node a (Tree a) (Tree a)
```

13.2. Recursión estructural

```
size Leaf = 0  
size (Node x l r) =  
  1 + size l + size r
```

Hasta aquí estudiamos transformaciones funcionales directas.

Sin embargo, la recursión sobre árboles genera múltiples computaciones pendientes simultáneamente.

14. Stacks explícitos

Observación 14.1. En estructuras no lineales un único acumulador ya no alcanza para representar el control restante de la computación.

Por lo tanto es necesario introducir:

- stacks explícitos,
- marcas de retorno,
- resultados parciales.

15. Tail recursion sobre árboles

```
size0 [] [n] = n  
  
size0 (Leaf:ts) rs =  
  size0 ts (0:rs)  
  
size0 ((Node x i d):ts) rs =  
  size0 (i:d:Mark:ts) rs  
  
size0 (Mark:ts) (nd:ni:rs) =  
  size0 ts ((1+nd+ni):rs)
```

16. Programa IMP para árboles

```
ps := Cons(t,Nil);
rs := Nil;

while ps do
  Cons(tope,ts) ->

  case tope of

    Leaf ->
      ps,rs := ts, Cons(0,rs)

    Node(x,i,d) ->
      ps := Cons(i,Cons(d,Cons(Mark,ts)))

    Mark ->
      case rs of
        Cons(nd,rs1) ->
          case rs1 of
            Cons(ni,rs2) ->
              m := nd + ni;
              m := S(m);
              ps,rs := ts, Cons(m,rs2)
```

17. Ejercicios

1. Implementar `map` en IMP.
2. Programar `foldr` en IMP.
3. Programar `foldl` en IMP.
4. Construir un programa IMP para factorial.
5. Mostrar equivalencia entre un `while` y una función tail recursive.
6. Implementar sustitución lambda usando stacks explícitos.
7. Escribir `foldl` usando `foldr`.
8. Escribir `foldr` usando `foldl`.

18. Referencias

Referencias

- [1] Álvaro Tasistro, *Eliminación de la Recursión*.
- [2] Richard Bird y Philip Wadler, *Introduction to Functional Programming*.
- [3] Simon Peyton Jones, *The Implementation of Functional Programming Languages*.
- [4] Friedman, Wand y Haynes, *Essentials of Programming Languages*.